

Task 1: System Provisioning & Linux Administration

Step 1: Creates a new system user named trainee using the adduser command, setting up its password and filling in user contact details (Full Name, Room Number, etc.)

```
ubuntu@devops-server:~$ sudo adduser trainee
[sudo: authenticate] Password:
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for trainee
Enter the new value, or press ENTER for the default
    Full Name []: trainee
    Room Number []: 1
    Work Phone []: 123456789
    Home Phone []: 0987654321
    Other []: 000000
Is the information correct? [Y/n] y
```

Step 2: Grants trainee administrative privileges by adding them to the sudo group using usermod, and then switches to the newly created user session using the su command

```
ubuntu@devops-server:~$ sudo usermod -aG sudo trainee
ubuntu@devops-server:~$ su - trainee
Password:
```

Step 3: Opens the SSH daemon configuration file (/etc/ssh/sshd_config) using the nano editor with sudo privileges to modify remote login settings.

```
trainee@devops-server:~$ sudo nano /etc/ssh/sshd_config
[sudo: authenticate] Password:
trainee@devops-server:~$ |
```

Step 4: Edits the configuration parameters inside sshd_config to change the default port from 22 to 2222 and explicitly sets PermitRootLogin no to block direct root access over SSH.

```
GNU nano 8.7.1 /etc/ssh/sshd_config
# ListenAddress.
#
# For changes to take effect, run:
#
#   systemctl daemon-reload
#   systemctl restart ssh.socket
#
#Port 22
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```
GNU nano 8.7.1 /etc/ssh/sshd_config *
# ListenAddress.
#
# For changes to take effect, run:
#
#   systemctl daemon-reload
#   systemctl restart ssh.socket
#
#Port 22
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```
GNU nano 8.7.1 /etc/ssh/sshd_config *
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
PermitRootLogin no
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2

#AuthorizedPrincipalsFile none
```

Step 5: Configures the Uncomplicated Firewall (UFW) by setting default policies (deny incoming, allow outgoing) and explicitly allowing TCP traffic on ports 2222 (SSH), 80 (HTTP), and 443 (HTTPS).

```
trainee@devops-server:~$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
trainee@devops-server:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
trainee@devops-server:~$ sudo ufw allow 2222/tcp
Rules updated
Rules updated (v6)
trainee@devops-server:~$ sudo ufw allow 80/tcp
Skipping adding existing rule
Skipping adding existing rule (v6)
trainee@devops-server:~$ sudo ufw allow 443/tcp
Rules updated
Rules updated (v6)
trainee@devops-server:~$ sudo ufw enable
Firewall is active and enabled on system startup
```

Step 6: Displays the active firewall rules using `sudo ufw status verbose` to verify that all ports are correctly secured and restricted.

```
trainee@devops-server:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
80/tcp ALLOW IN Anywhere
Anywhere DENY IN 10.0.0.99
2222/tcp ALLOW IN Anywhere
443/tcp ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)
80/tcp (v6) ALLOW IN Anywhere (v6)
2222/tcp (v6) ALLOW IN Anywhere (v6)
443/tcp (v6) ALLOW IN Anywhere (v6)

trainee@devops-server:~$
```

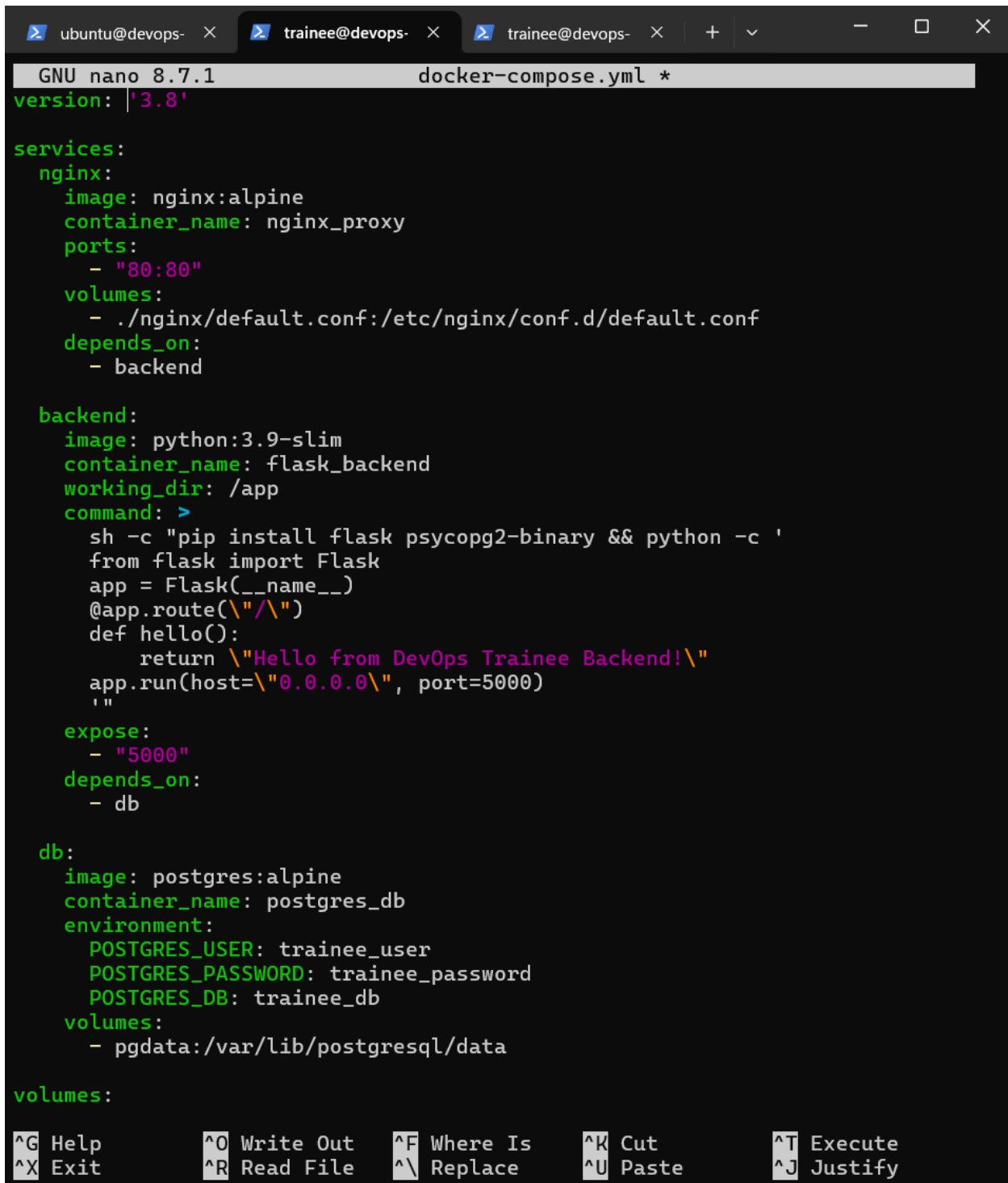
Task 2: Containerization & Web Services

Step 1: Creates a dedicated project directory called `web-stack` with an internal subfolder for `nginx` configurations using `mkdir` and navigates into it using `cd`.

```
trainee@devops-server:~$ mkdir -p ~/web-stack/nginx
trainee@devops-server:~$ ls
web-stack
trainee@devops-server:~$ cd web-stack/
trainee@devops-server:~/web-stack$ ls
nginx
trainee@devops-server:~/web-stack$ |
```

Step 2: Opens and edits the docker-compose.yml file to orchestrate three distinct services: Nginx (reverse proxy), a Python Flask backend, and a PostgreSQL database

```
trainee@devops-server:~/web-stack$ nano docker-compose.yml
trainee@devops-server:~/web-stack$ |
```



```
ubuntu@devops-  X  trainee@devops-  X  trainee@devops-  X  +  -  □  X
GNU nano 8.7.1      docker-compose.yml *
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    container_name: nginx_proxy
    ports:
      - "80:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
    depends_on:
      - backend

  backend:
    image: python:3.9-slim
    container_name: flask_backend
    working_dir: /app
    command: >
      sh -c "pip install flask psycopg2-binary && python -c '
      from flask import Flask
      app = Flask(__name__)
      @app.route(\"/\")
      def hello():
          return \"Hello from DevOps Trainee Backend!\"
      app.run(host=\"0.0.0.0\", port=5000)
      \"
    expose:
      - "5000"
    depends_on:
      - db

  db:
    image: postgres:alpine
    container_name: postgres_db
    environment:
      POSTGRES_USER: trainee_user
      POSTGRES_PASSWORD: trainee_password
      POSTGRES_DB: trainee_db
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^\\ Replace	^U Paste	^J Justify

Step 3: Generates the Flask application script (app.py) which returns the required verification message, and fine-tunes the service dependencies and volume mounts inside the compose configuration.

```
trainee@devops-server:~/web-stack$ cat << 'EOF' > app.py
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello():
    return "Hello from DevOps Trainee Backend!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
EOF
trainee@devops-server:~/web-stack$ nano docker-compose.yml
trainee@devops-server:~/web-stack$ |
```

```
GNU nano 8.7.1 docker-compose.yml *
services:
  nginx:
    image: nginx:alpine
    container_name: nginx_proxy
    ports:
      - "80:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
    depends_on:
      - backend
  backend:
    image: python:3.9-slim
    container_name: flask_backend
    working_dir: /app
    volumes:
      - ./app
    command: sh -c "pip install flask psycopg2-binary && python app.py"
    expose:
      - "5000"
    depends_on:
      - db
  db:
    image: postgres:alpine
    container_name: postgres_db
    environment:
      POSTGRES_USER: trainee_user
      POSTGRES_PASSWORD: trainee_password
      POSTGRES_DB: trainee_db
    volumes:
      - pgdata:/var/lib/postgresql
volumes:
  pgdata:
```

Step 4: Edits the custom Nginx server configuration (nginx/default.conf) to listen on port 80 and route incoming HTTP requests to the internal Flask backend container (http://flask_backend:5000).

```
trainee@devops-server:~/web-stack$ nano nginx/default.conf
trainee@devops-server:~/web-stack$ |
```

```
GNU nano 8.7.1
server {
    listen 80;
    server_name localhost;

    location / {
        proxy_pass http://flask_backend:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Step 5: Runs docker ps to verify that all containers (nginx_proxy, flask_backend, postgres_db, and an existing jenkins container) are up and running with their correct port mappings.

```
trainee@devops-server:~/web-stack$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
679d361dcee6   nginx:alpine   "/docker-entrypoint..." 9 seconds ago  Up 6 seconds  0.0.0.0:80->80/tcp, [::]:80->80/tcp  nginx_proxy
3a8e0b7ec555   python:3.9-slim "sh -c 'pip install ..." 9 seconds ago  Up 7 seconds  5000/tcp                            flask_backend
c3d10767282b   postgres:alpine "docker-entrypoint.s..." 10 seconds ago Up 8 seconds  5432/tcp                            postgres_db
5775fb78630b   jenkins/jenkins:lts "/usr/bin/tini -- /u..." 2 weeks ago   Up 7 hours                                jenkins
trainee@devops-server:~/web-stack$ |
```

Step 6: Tests the reverse proxy routing by executing a `curl http://localhost/` command, which successfully returns the response message from the Flask backend via Nginx.

```
trainee@devops-server:~/web-stack$ curl http://localhost/  
Hello from DevOps Trainee Backend!trainee@devops-server:~/web-stack$  
trainee@devops-server:~/web-stack$ |
```

Task 3: Automation & Shell Scripting

Step 1: Creates and populates the `infra_health_check.sh` script to verify both the Nginx-Flask web routing and the active running status of all core Docker containers.

```
trainee@devops-server:~/web-stack$ cat << 'EOF' > infra_health_check.sh  
#!/bin/bash  
  
echo "=== Infrastructure Health Check ==="  
  
# Check Nginx status  
if curl -s http://localhost/ | grep -q "Hello from DevOps Trainee Backend"; then  
echo "[PASS] Nginx proxy and Flask backend are responding correctly."  
else  
echo "[FAIL] Nginx proxy or Flask backend is down."  
fi  
  
# Check Docker container running status  
containers=("nginx_proxy" "flask_backend" "postgres_db")  
for container in "${containers[@]"; do  
if [ "$(sudo docker inspect -f '{{.State.Running}}' "$container" 2>/dev/null)" == "true" ]; then  
echo "[PASS] Container $container is running."  
else  
echo "[FAIL] Container $container is NOT running."  
fi  
done  
EOF  
trainee@devops-server:~/web-stack$ cat infra_health_check.s  
cat: infra_health_check.s: No such file or directory  
trainee@devops-server:~/web-stack$ cat infra_health_check.sh  
#!/bin/bash  
  
echo "=== Infrastructure Health Check ==="  
  
# Check Nginx status  
if curl -s http://localhost/ | grep -q "Hello from DevOps Trainee Backend"; then  
echo "[PASS] Nginx proxy and Flask backend are responding correctly."  
else  
echo "[FAIL] Nginx proxy or Flask backend is down."  
fi  
  
# Check Docker container running status  
containers=("nginx_proxy" "flask_backend" "postgres_db")  
for container in "${containers[@]"; do  
if [ "$(sudo docker inspect -f '{{.State.Running}}' "$container" 2>/dev/null)" == "true" ]; then  
echo "[PASS] Container $container is running."  
else  
echo "[FAIL] Container $container is NOT running."  
fi  
done
```

Step 2: Assigns execution permissions to the script using `chmod +x` and verifies file details with `ls -l`.

```
trainee@devops-server:~/web-stack$ chmod +x infra_health_check.sh
trainee@devops-server:~/web-stack$ ls -l infra_health_check.sh
-rwxrwxr-x 1 trainee docker 643 Sep 10 13:43 infra_health_check.sh
trainee@devops-server:~/web-stack$ |
```

Step 3: Executes the script locally (`./infra_health_check.sh`) to ensure it successfully outputs [PASS] statuses for the services.

```
trainee@devops-server:~/web-stack$ ./infra_health_check.sh
=== Infrastructure Health Check ===
[PASS] Nginx proxy and Flask backend are responding correctly.
[PASS] Container nginx_proxy is running.
[PASS] Container flask_backend is running.
[PASS] Container postgres_db is running.
```

Step 4: Opens the cron table editor using `crontab -e` and selects the nano text editor to configure automated task scheduling.

```
trainee@devops-server:~/web-stack$ crontab -e
no crontab for trainee - using an empty one
Select an editor. To change later, run select-editor again.
 1. /bin/nano          <---- easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny
 4. /bin/ed

Choose 1-4 [1]: 1
crontab: installing new crontab
trainee@devops-server:~/web-stack$ |
```

Step 5: Adds a cron schedule rule (* / 15 * * * *) to execute the health check script automatically every 15 minutes, piping outputs and errors into a log file (health_check.log).

```
ubuntu@devops-server: ~/do: x  trainee@devops-server: ~/we x  trainee@devops-server: ~ x + v
GNU nano 8.7.1 /tmp/crontab.gF5FTm/crontab *
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
0/15 * * * * /home/trainee/web-stack/infra_health_check.sh >> /home/trainee/web-stack/health_check.log 2>&1
```

Step 6: Verifies that the cron job has been successfully installed and registered for the user using crontab -l.

```
trainee@devops-server:~/web-stack$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
*/15 * * * * /home/trainee/web-stack/infra_health_check.sh >> /home/trainee/web-stack/health_check.log 2>&1
trainee@devops-server:~/web-stack$ |
```

Task 4: Monitoring, Backups & Disaster Recovery

Step 1: Creates the backup directory structure at /var/backups/db and writes the db_backup.sh script, which uses pg_dump inside the PostgreSQL container and compresses the output using gzip with a dynamic date timestamp.

```
trainee@devops-server:~/web-stack$ sudo mkdir -p /var/backups/db
[sudo: authenticate] Password:

trainee@devops-server:~/web-stack$ cat << 'EOF' > db_backup.sh
#!/bin/bash

BACKUP_DIR="/var/backups/db"
TIMESTAMP=$(date +"%Y%m%d")
ARCHIVE_NAME="db_backup_${TIMESTAMP}.sql.gz"

# Ensure backup directory exists
sudo mkdir -p "$BACKUP_DIR"

# Dump PostgreSQL from container and compress
sudo docker exec -t postgres_db pg_dump -U postgres postgres | gzip > "$BACKUP_DIR/$ARCHIVE_NAME"

echo "[SUCCESS] Database backup saved to $BACKUP_DIR/$ARCHIVE_NAME"
EOF
```

Step 2: Fixes permission issues using chmod +x, runs the backup script with sudo, and lists the contents of the backup directory to confirm the .sql.gz archive is successfully saved.

```
trainee@devops-server:~/web-stack$ gunzip < /var/backups/db/db_backup_YYYYMMDD.sql.gz | sudo docker exec -i postgres_db psql -U postgres -d postgres
bash: /var/backups/db/db_backup_YYYYMMDD.sql.gz: No such file or directory
psql: error: connection to server on socket "/var/run/postgresql/.s.PGSQL.5432" failed: FATAL: role "postgres" does not exist
trainee@devops-server:~/web-stack$ sudo ./db_backup.sh
sudo: cannot execute '/home/trainee/web-stack/db_backup.sh': Permission denied (os error 13)
trainee@devops-server:~/web-stack$ chmod +x db_backup.sh
trainee@devops-server:~/web-stack$ sudo ./db_backup.sh
[sudo: authenticate] Password:
[SUCCESS] Database backup saved to /var/backups/db/db_backup_20260910.sql.gz
```

```
trainee@devops-server:~/web-stack$ ls -l /var/backups/db/
total 4
-rw-r--r-- 1 root root 135 Sep 10 15:41 db_backup_20260910.sql.gz
trainee@devops-server:~/web-stack$
```

Step 3: Tests the database recovery process by piping the unzipped backup archive back into the PostgreSQL container (psql) to ensure data integrity.

```
trainee@devops-server:~/web-stack$ sudo ./db_backup.sh
gunzip < /var/backups/db/db_backup_20260910.sql.gz | sudo docker exec -i postgres_db psql -U trainee_user -d trainee_db
SET
SET
SET
SET
SET
SET
SET
   set_config
-----
(1 row)

SET
SET
SET
SET
```

Step 4: Validates database restoration by connecting to PostgreSQL, creating a sample verification table (items), checking database schema tables via \dt, and grepping the compressed backup archive to confirm structural data exists

```
trainee@devops-server:~/web-stack$ sudo docker exec -it postgres_db psql -U trainee_user -d trainee_db -c "CREATE TABLE items (id SERIAL PRIMARY KEY, name VARCHAR(50));"
CREATE TABLE
trainee@devops-server:~/web-stack$ sudo docker exec -it postgres_db psql -U trainee_user -d trainee_db -c "\dt"
      List of tables
 Schema | Name  | Type  | Owner
-----+-----+-----+-----
 public | items | table | trainee_user
(1 row)
```

```

trainee@devops-server:~/web-stack$ sudo ./db_backup.sh
trainee@devops-server:~/web-stack$ gunzip -c /var/backups/db/db_backup_20260910.sql.gz | grep items
-- Name: items; Type: TABLE; Schema: public; Owner: trainee_user
CREATE TABLE public.items (
ALTER TABLE public.items OWNER TO trainee_user;
-- Name: items_id_seq; Type: SEQUENCE; Schema: public; Owner: trainee_user
CREATE SEQUENCE public.items_id_seq
ALTER SEQUENCE public.items_id_seq OWNER TO trainee_user;
-- Name: items_id_seq; Type: SEQUENCE OWNED BY; Schema: public; Owner: trainee_user
ALTER SEQUENCE public.items_id_seq OWNED BY public.items.id;
-- Name: items id; Type: DEFAULT; Schema: public; Owner: trainee_user
ALTER TABLE ONLY public.items ALTER COLUMN id SET DEFAULT nextval('public.items_id_seq'::regclass);
-- Data for Name: items; Type: TABLE DATA; Schema: public; Owner: trainee_user
COPY public.items (id, name) FROM stdin;
-- Name: items_id_seq; Type: SEQUENCE SET; Schema: public; Owner: trainee_user
SELECT pg_catalog.setval('public.items_id_seq', 1, false);
-- Name: items_items_pkey; Type: CONSTRAINT; Schema: public; Owner: trainee_user
ALTER TABLE ONLY public.items
    ADD CONSTRAINT items_pkey PRIMARY KEY (id);
trainee@devops-server:~/web-stack$

```

Step 5: Updates `infra_health_check.sh` to include system-level resource checks, capturing disk space usage (`df`), memory allocation (`free`), and container resource performance metrics (`docker stats`).

```

trainee@devops-server:~/web-stack$ nano infra_health_check.sh

```

```

GNU nano 8.7.1 infra_health_check.sh
#!/bin/bash

echo "=== Infrastructure Health & Metrics Report: $(date) ==="

# Check Nginx status
if curl -s http://localhost/ | grep -q "Hello from DevOps Trainee Backend"; then
    echo "[PASS] Nginx proxy and Flask backend are responding correctly."
else
    echo "[FAIL] Nginx proxy or Flask backend is down."
fi

# Check Docker container running status
containers=("nginx_proxy" "flask_backend" "postgres_db")
for container in "${containers[@]}; do
    if [ "$(sudo docker inspect -f '{{.State.Running}}' "$container" 2>/dev/null)" == "true" ]; then
        echo "[PASS] Container $container is running."
    else
        echo "[FAIL] Container $container is NOT running."
    fi
done

# Basic System & Container Metrics
echo "--- System Resource Metrics ---"
echo "Disk Usage:"
df -h / | awk 'NR==1 || NR==2 {print $0}'

echo "Memory Usage:"
free -h

echo "Container Resource Utilization:"
sudo docker stats --no-system --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"

```

Step 6: Executes the updated health check script and reviews the recent lines of the health_check.log file using tail to confirm that CPU, memory, and container stats are accurately logged.

```
trainee@devops-server:~/web-stack$ cat << 'EOF' > infra_health_check.sh
#!/bin/bash

LOG_FILE="/home/trainee/web-stack/health_check.log"

echo "=== Infrastructure Health & Metrics Report: $(date) ===" >> "$LOG_FILE"

# Check Nginx status
if curl -s http://localhost/ | grep -q "Hello from DevOps Trainee Backend"; then
    echo "[PASS] Nginx proxy and Flask backend are responding correctly." >> "$LOG_FILE"
else
    echo "[FAIL] Nginx proxy or Flask backend is down." >> "$LOG_FILE"
fi

# Check Docker container running status
containers=("nginx_proxy" "flask_backend" "postgres_db")
for container in "${containers[@]}; do
    if [ "$(docker inspect -f '{{.State.Running}}' "$container" 2>/dev/null)" == "true" ]; then
        echo "[PASS] Container $container is running." >> "$LOG_FILE"
    else
        echo "[FAIL] Container $container is NOT running." >> "$LOG_FILE"
    fi
done

# Basic System & Container Metrics
echo "--- System Resource Metrics ---" >> "$LOG_FILE"
echo "Disk Usage:" >> "$LOG_FILE"
df -h / | awk 'NR==1 || NR==2 {print $0}' >> "$LOG_FILE"

echo "Memory Usage:" >> "$LOG_FILE"
free -h >> "$LOG_FILE"

echo "Container Resource Utilization:" >> "$LOG_FILE"
docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}" >> "$LOG_FILE"
echo "" >> "$LOG_FILE"
EOF
trainee@devops-server:~/web-stack$ chmod +x infra_health_check.sh
```

```

trainee@devops-server:~/web-stack$ chmod +x infra_health_check.sh
sudo ./infra_health_check.sh
=== Infrastructure Health & Metrics Report: Thu Sep 10 04:21:16 PM UTC 2026 ===
[PASS] Nginx proxy and Flask backend are responding correctly.
[PASS] Container nginx_proxy is running.
[PASS] Container flask_backend is running.
[PASS] Container postgres_db is running.
--- System Resource Metrics ---
Disk Usage:
Filesystem                                Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv        23G   16G   5.9G  73% /
Memory Usage:
      total        used        free      shared  buff/cache   available
Mem:    3.3Gi       1.5Gi       229Mi        40Mi       1.9Gi       1.8Gi
Swap:    2.0Gi       657Mi       1.4Gi
Container Resource Utilization:
unknown flag: --no-system

Usage:  docker stats [OPTIONS] [CONTAINER...]

Run 'docker stats --help' for more information

```

```

trainee@devops-server:~/web-stack$ tail -n 25 health_check.log
      total        used        free      shared  buff/cache   available
Mem:    3.3Gi       1.5Gi       215Mi        40Mi       1.9Gi       1.8Gi
Swap:    2.0Gi       657Mi       1.4Gi
Container Resource Utilization:
sudo: A terminal is required to authenticate
=== Infrastructure Health & Metrics Report: Thu Sep 10 04:57:23 PM UTC 2026 ===
[PASS] Nginx proxy and Flask backend are responding correctly.
[PASS] Container nginx_proxy is running.
[PASS] Container flask_backend is running.
[PASS] Container postgres_db is running.
--- System Resource Metrics ---
Disk Usage:
Filesystem                                Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv        23G   16G   5.9G  73% /
Memory Usage:
      total        used        free      shared  buff/cache   available
Mem:    3.3Gi       1.5Gi       223Mi        40Mi       1.9Gi       1.8Gi
Swap:    2.0Gi       657Mi       1.4Gi
Container Resource Utilization:
NAME           CPU %      MEM USAGE / LIMIT
nginx_proxy    0.00%     2.629MiB / 3.319GiB
flask_backend  0.05%     23.99MiB / 3.319GiB
postgres_db    0.00%     33.66MiB / 3.319GiB
jenkins        0.38%     304.8MiB / 3.319GiB
trainee@devops-server:~/web-stack$ |

```

Task 5: Git & Documentation

Step 1: Creates a .gitignore file to exclude sensitive environment variables (.env), database backups (/var/backups/db/), log files, and local IDE directories from version control.

```
trainee@devops-server:~/web-stack$ cat << 'EOF' > .gitignore
# Environment & Secrets
.env
*.env

# Backups & Logs
/var/backups/db/
*.sql.gz
health_check.log

# Python / IDE
__pycache__/
*.pyc
.venv/
venv/
.idea/
.vscode/
EOF
```

Step 2: Runs git status to check untracked project configuration files prior to committing.

```
trainee@devops-server:~/web-stack$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .gitignore
    app.py
    db_backup.sh
    docker-compose.yml
    infra_health_check.sh
    nginx/

nothing added to commit but untracked files present (use "git add" to track)
```

Step 3: Runs git status to check untracked project configuration files prior to committing.

```
trainee@devops-server:~/web-stack$ nano README.md
trainee@devops-server:~/web-stack$ |
```

Step 4: Configures local Git user credentials (user.email and user.name), commits all project files to the master branch, and verifies the commit history using git log.

```
trainee@devops-server:~/web-stack$ git init
Reinitialized existing Git repository in /home/trainee/web-stack/.git/
trainee@devops-server:~/web-stack$ git add README.md .gitignore infra_health_check.sh db_backup.sh app.py docker-compose.yml
trainee@devops-server:~/web-stack$ git commit -m "Added project and configuration files"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (not 'trainee@devops-server (none)')
```

```
ubuntu@devops-server:~/devops-stack$ nano README.md
# DevOps Stack & Deployment Project

## 1. Architecture Overview
This project deploys a multi-container web application stack using Docker and Docker Compose.

### Stack Components:
- **Nginx Proxy ('nginx_proxy')**: Acts as the reverse proxy, routing incoming HTTP traffic on port 80 to the backend application.
- **Flask Backend ('flask_backend')**: Python/Flask application running on port 5000, serving API/web responses.
- **PostgreSQL Database ('postgres_db')**: Relational database running on port 5432 with persistent volume storage.
- **Jenkins ('jenkins')**: Self-hosted CI/CD automation server.

---

## 2. Backup & Disaster Recovery
Automated database backups are configured via a shell script stored in the project directory.

- **Backup Script ('db_backup.sh')**: Executes a 'pg_dump' inside the PostgreSQL container, compresses the file using 'gzip', and stores it with a timestamp.
- **Storage Path**: '/var/backups/db/'
- **Naming Convention**: 'db_backup_YYYYMMDD.sql.gz'

### Restoration Workflow:
To restore the database from a backup archive, run:
```bash
gunzip -c /var/backups/db/db_backup_YYYYMMDD.sql.gz | docker exec -i postgres_db psql -U trainee_user -d trainee_db
```

```
trainee@devops-server:~/web-stack$ git config user.email "luitelbeevaa@gmail.com"
trainee@devops-server:~/web-stack$ git config user.name "Biva Luitel"
trainee@devops-server:~/web-stack$ git commit -m "Added project and configuration files"
[master (root-commit) 37b59d4] Added project and configuration files
 6 files changed, 127 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 README.md
 create mode 100644 app.py
 create mode 100755 db_backup.sh
 create mode 100644 docker-compose.yml
 create mode 100755 infra_health_check.sh
trainee@devops-server:~/web-stack$ git log -n 1
commit 37b59d46d40d92649e6c17a0de41cddb9c75b3ce (HEAD -> master)
Author: Biva Luitel <luitelbeevaa@gmail.com>
Date: Thu Sep 10 17:27:25 2026 +0000

 Added project and configuration files
trainee@devops-server:~/web-stack$ |
```